

Kmeans291125

November 29, 2025

```
[1]: # Paso 1 Librerías y carga de datos
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb
from sklearn.cluster import KMeans
from sklearn.metrics import pairwise_distances_argmin_min
from mpl_toolkits.mplot3d import Axes3D
plt.rcParams['figure.figsize'] = (16,9)
plt.style.use('ggplot')
```

```
[2]: #Carga de datos
data = pd.read_csv(" analisis.csv")
```

```
[3]: # Paso 2 Análisis exploratorio de los datos
data.head()
```

```
[3]:
```

	usuario	op	co	ex	ag	ne \
0	3gerardpique	34.297953	28.148819	41.948819	29.370315	9.841575
1	aguerosergiokun	44.986842	20.525865	37.938947	24.279098	10.362406
2	albertochicote	41.733854	13.745417	38.999896	34.645521	8.836979
3	AlejandroSanz	40.377154	15.377462	52.337538	31.082154	5.032231
4	alfredocasero1	36.664677	19.642258	48.530806	31.138871	7.305968

	wordcount	categoria
0	37.0945	7
1	78.7970	7
2	49.2604	4
3	80.4538	2
4	47.0645	4

```
[4]: data.describe()
```

```
[4]:
```

	op	co	ex	ag	ne	wordcount \
count	140.000000	140.000000	140.000000	140.000000	140.000000	140.000000
mean	44.414591	22.977135	40.764428	22.918528	8.000098	98.715484
std	8.425723	5.816851	7.185246	7.657122	3.039248	44.714071
min	30.020465	7.852756	18.693542	9.305985	1.030213	5.020800

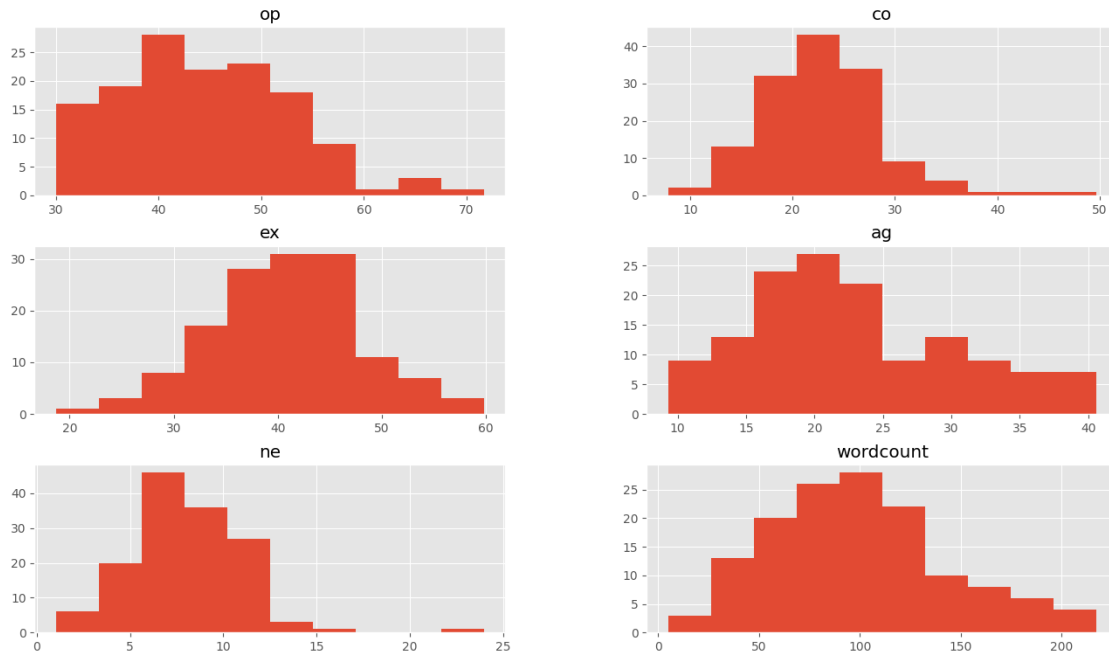
25%	38.206484	19.740299	36.095722	17.050993	6.086144	66.218475
50%	44.507091	22.466718	41.457492	21.384554	7.839722	94.711400
75%	49.365923	26.091606	45.197769	28.678866	9.758189	119.707925
max	71.696129	49.637863	59.824844	40.583162	23.978462	217.183200

	categoria
count	140.000000
mean	4.050000
std	2.658839
min	1.000000
25%	2.000000
50%	3.500000
75%	7.000000
max	9.000000

```
[5]: data.groupby('categoria').size()
```

```
[5]: categoria
1      27
2      34
3       9
4      19
5       4
6       8
7      17
8      16
9       6
dtype: int64
```

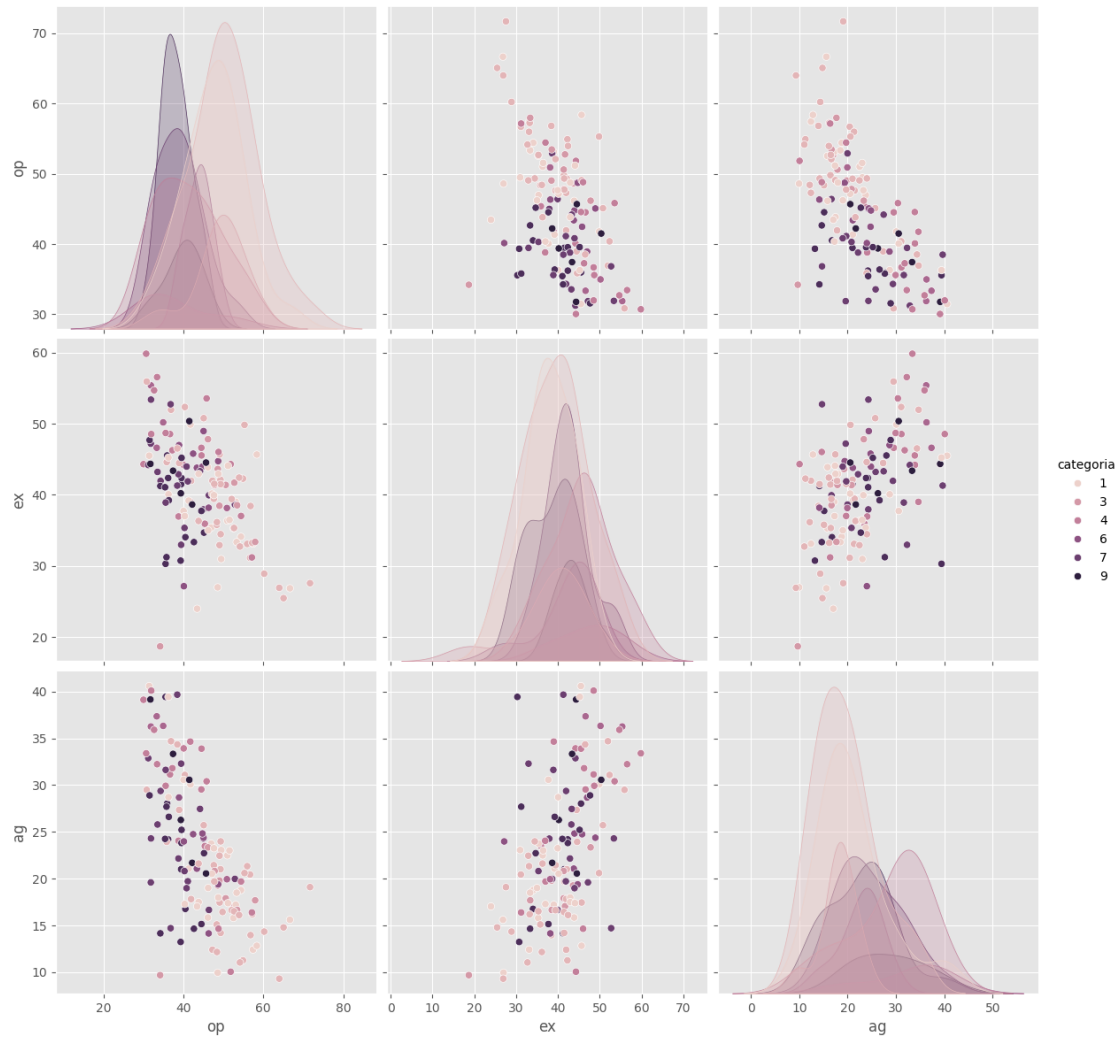
```
[6]: data.drop(['categoria'], axis = 1).hist()
plt.show()
```



```
[7]: sb.pairplot(data.dropna(), hue='categoria', size=4, vars=['op', 'ex', 'ag'],
      kind='scatter')
```

/Users/haydeml/Library/Python/3.9/lib/python/site-packages/seaborn/axisgrid.py:2100: UserWarning: The `size` parameter has been renamed to `height`; please update your code.
warnings.warn(msg, UserWarning)

```
[7]: <seaborn.axisgrid.PairGrid at 0x107464640>
```



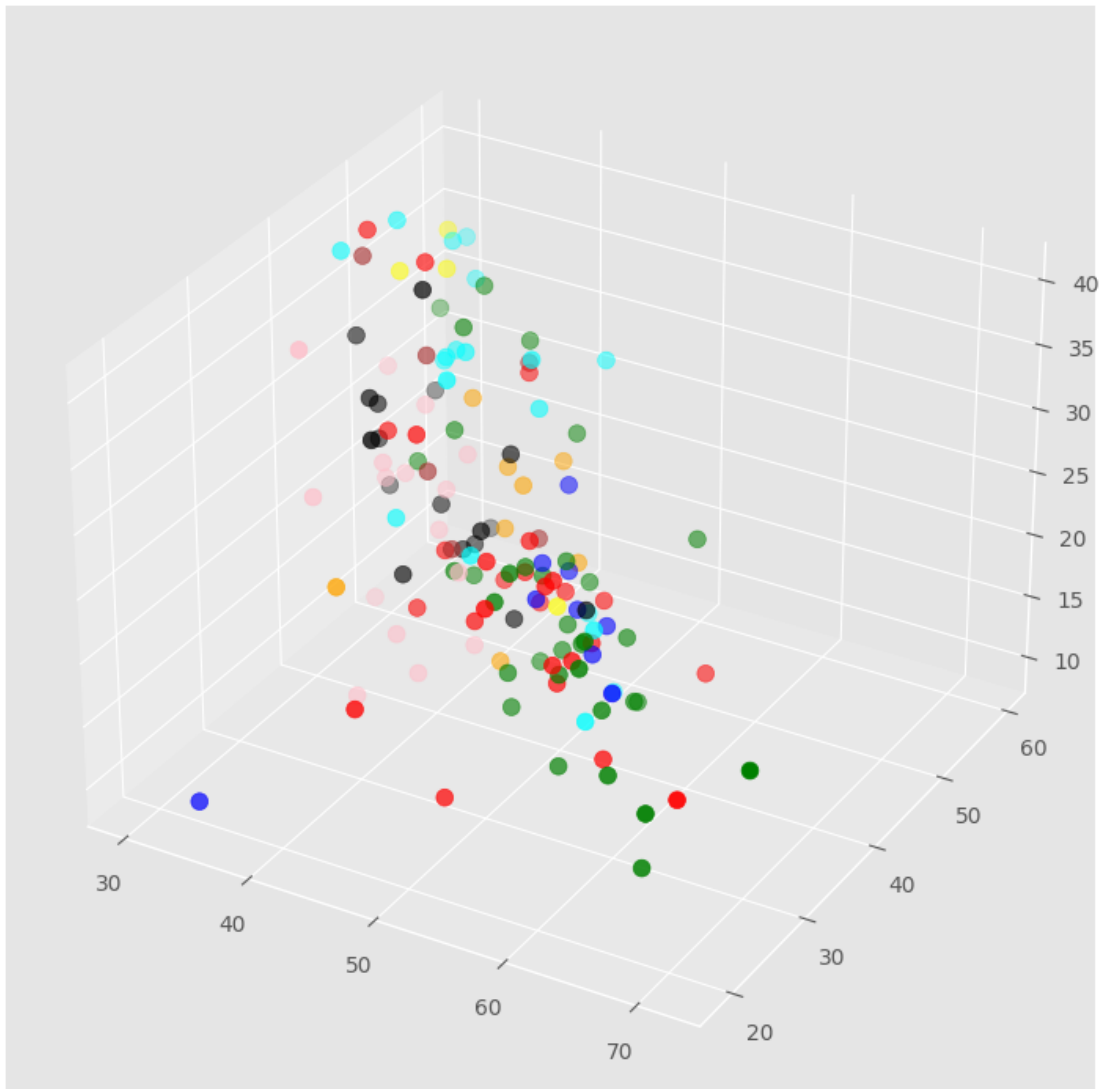
```
[9]: # Paso 3 separación de datos y entrenamiento
X = np.array(data[['op', 'ex', 'ag']])
y = np.array(data['categoria'])
X.shape
```

```
[9]: (140, 3)
```

```
[10]: fig = plt.figure()
ax = fig.add_subplot(111, projection='3d') # Configuración correcta del
↳ gráfico 3D
colores = ['blue', 'red', 'green', 'blue', 'cyan', 'yellow', 'orange',
           'black', 'pink', 'brown', 'purple']
asignar = []
for row in y:
    asignar.append(colores[row % len(colores)]) # Evitar errores de índice
```

```
# Graficar los puntos
ax.scatter(X[:, 0], X[:, 1], X[:, 2], c=asignar, s=60)

# Mostrar la gráfica
plt.show()
```



```
[12]: Nc = range(1,20)
kmeans = [KMeans(n_clusters=i) for i in Nc]
kmeans
```

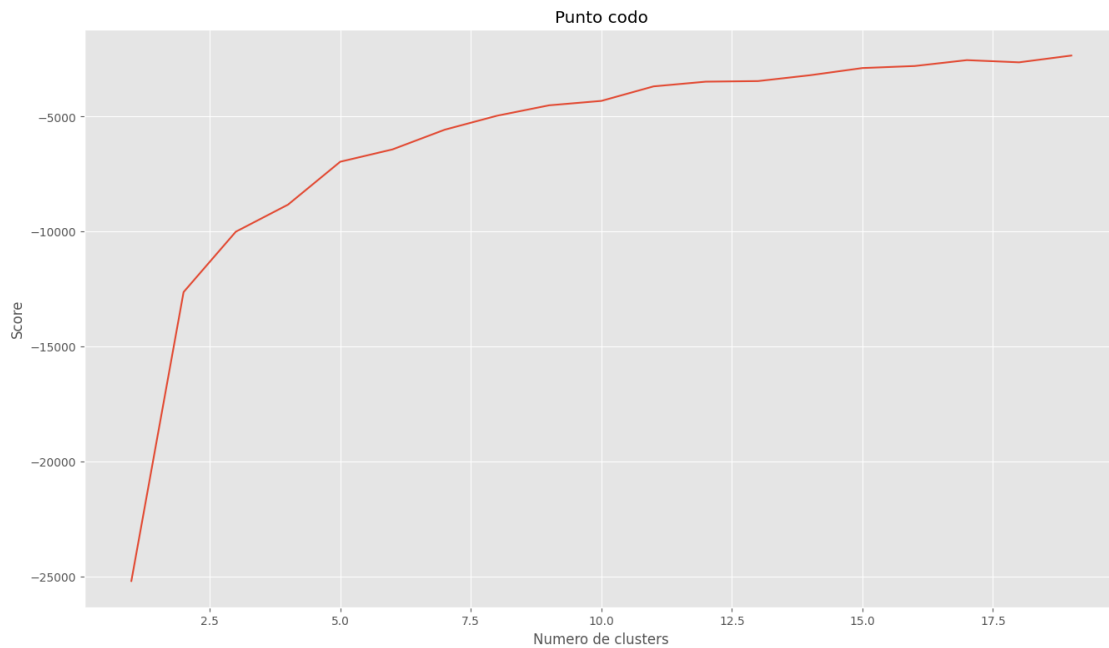
```
[12]: [KMeans(n_clusters=1),
      KMeans(n_clusters=2),
      KMeans(n_clusters=3),
      KMeans(n_clusters=4),
      KMeans(n_clusters=5),
      KMeans(n_clusters=6),
      KMeans(n_clusters=7),
      KMeans(),
      KMeans(n_clusters=9),
      KMeans(n_clusters=10),
      KMeans(n_clusters=11),
      KMeans(n_clusters=12),
      KMeans(n_clusters=13),
      KMeans(n_clusters=14),
      KMeans(n_clusters=15),
      KMeans(n_clusters=16),
      KMeans(n_clusters=17),
      KMeans(n_clusters=18),
      KMeans(n_clusters=19)]
```

```
[14]: score = [kmeans[i].fit(X).score(X) for i in range(len(kmeans))]
      score
```

```
[14]: [-25194.03935276632,
      -12632.031536945231,
      -10006.303869646543,
      -8828.521443378128,
      -6962.912846887289,
      -6427.393950758512,
      -5570.8483261543315,
      -4964.071149968588,
      -4509.235167565541,
      -4317.608429043745,
      -3685.9301785499883,
      -3480.9904749477646,
      -3455.4462839330404,
      -3200.2532225370005,
      -2888.5568326208504,
      -2800.317884103801,
      -2544.4856057458755,
      -2640.00268409846,
      -2345.5966657475815]
```

```
[15]: plt.plot(Nc, score)
      plt.xlabel('Numero de clusters')
      plt.ylabel('Score')
      plt.title('Punto codo')
```

```
plt.show()
```



```
[17]: kmeans = KMeans(n_clusters=5).fit(X)
```

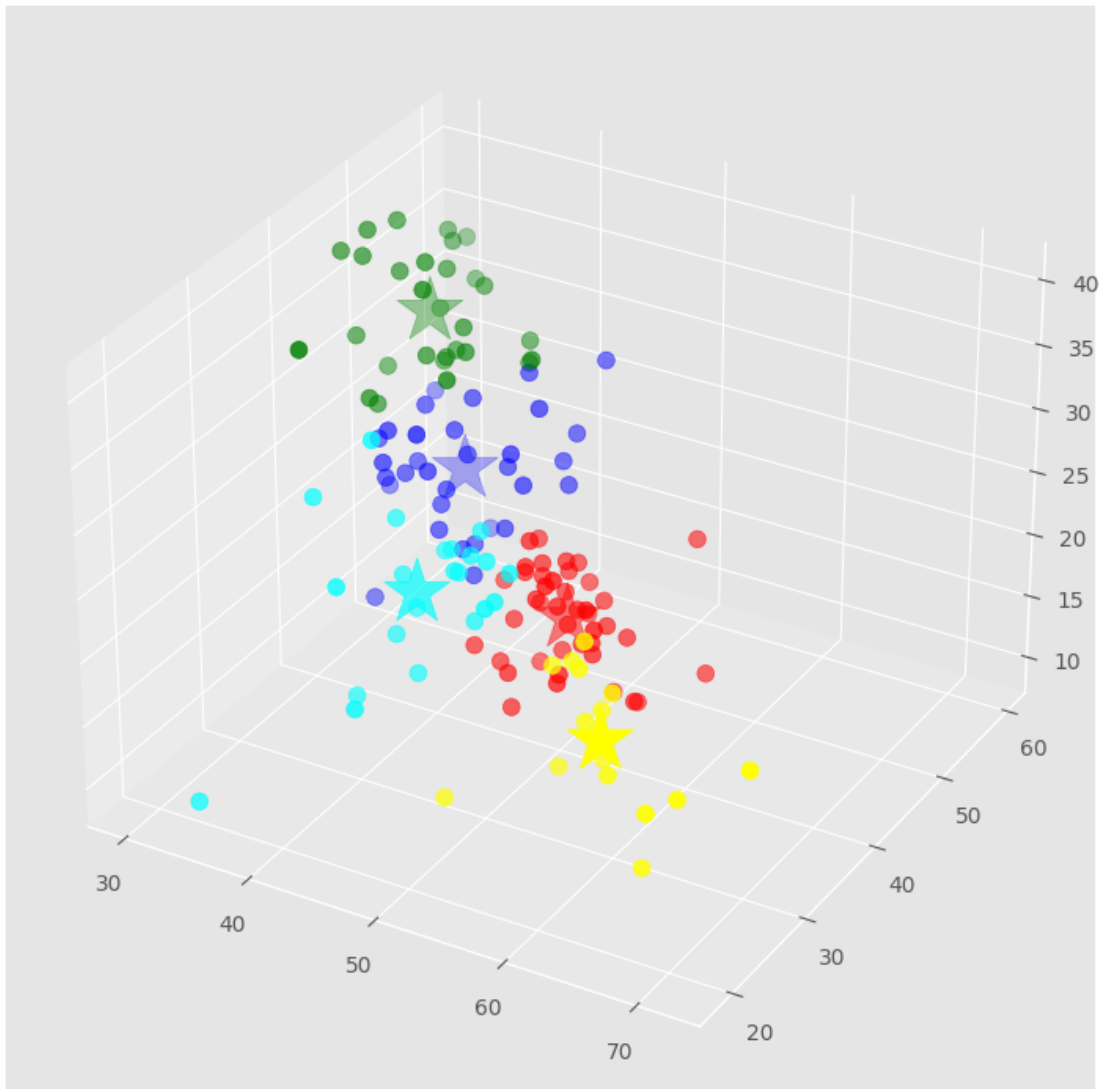
```
[19]: centroides = kmeans.cluster_centers_  
print(centroides)
```

```
[[50.15530371 40.81295548 17.39048745]  
 [35.39154169 47.17372597 34.50044186]  
 [39.55874337 44.81190669 24.291027  ]  
 [42.66891482 33.31008136 21.11530482]  
 [58.70462307 30.53566167 15.72207033]]
```

```
[20]: # Asignar colores a los clusters  
labels = kmeans.predict(X)  
C = kmeans.cluster_centers_ # Centros de los clusters  
colores = ['red', 'green', 'blue', 'cyan', 'yellow']  
asignar = [colores[row % len(colores)] for row in labels] # Asegurar que no  
↳excedan los índices  
  
# Crear la figura y el gráfico 3D  
fig = plt.figure()  
ax = fig.add_subplot(111, projection='3d') # Configuración recomendada  
  
# Graficar los puntos  
ax.scatter(X[:, 0], X[:, 1], X[:, 2], c=asignar, s=60)
```

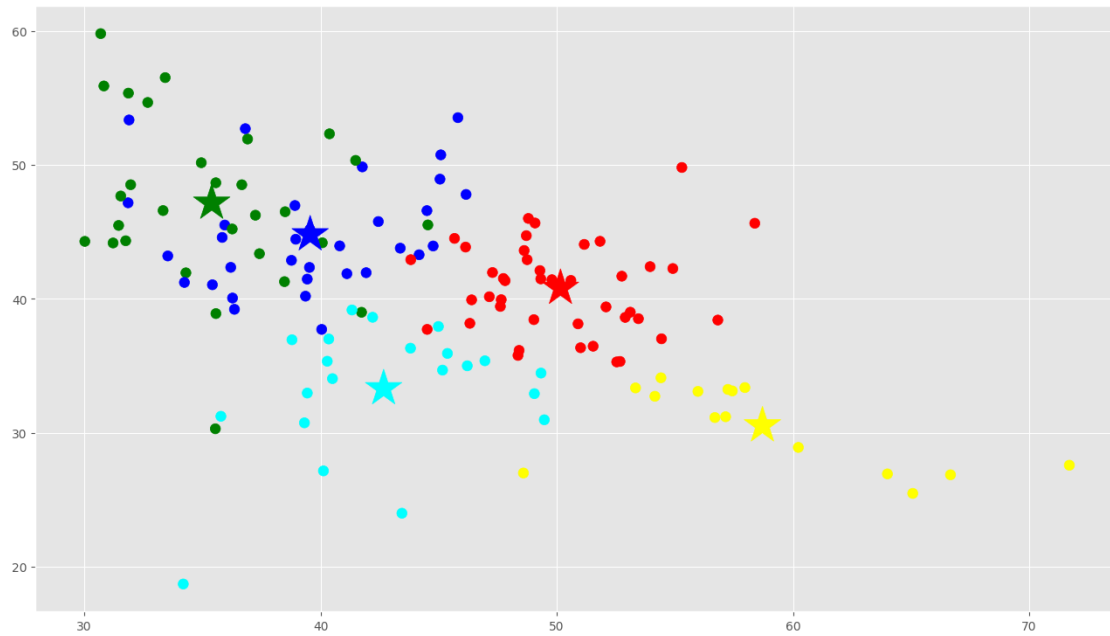
```
# Graficar los centroides
ax.scatter(C[:, 0], C[:, 1], C[:, 2], marker='*', c=colores[:len(C)], s=1000)

# Mostrar la gráfica
plt.show()
```



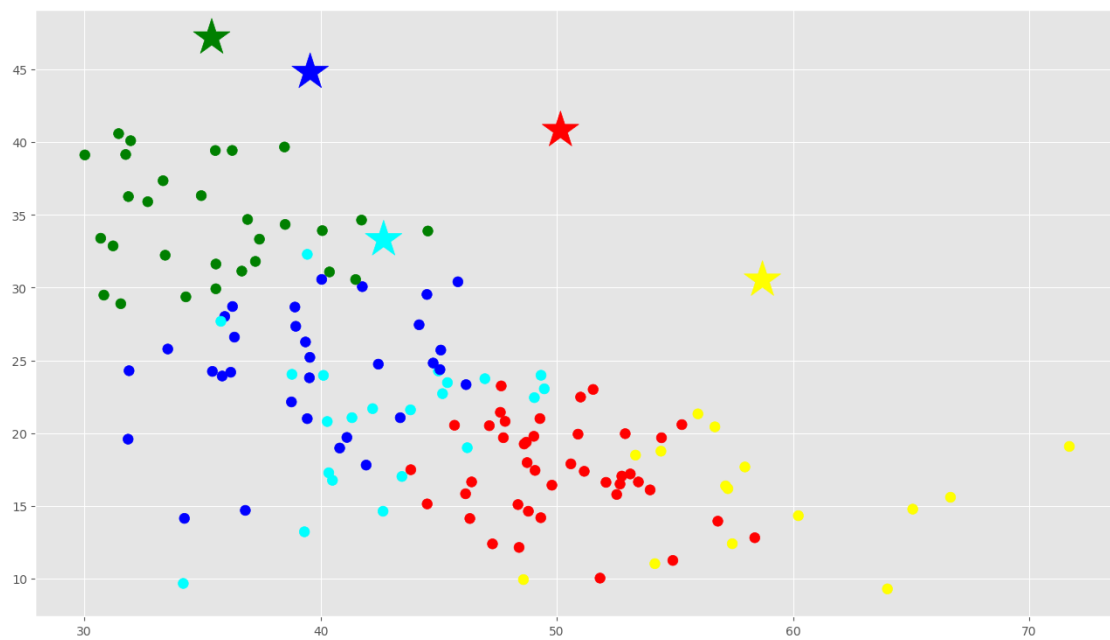
```
[25]: f1 = data['op'].values
      f2 = data['ex'].values
      plt.scatter(f1,f2, c=asignar, s=70)
      plt.scatter(C[:,0], C[:,1], marker="*", c=colores, s=1000)
```

```
[25]: <matplotlib.collections.PathCollection at 0x31db67dc0>
```

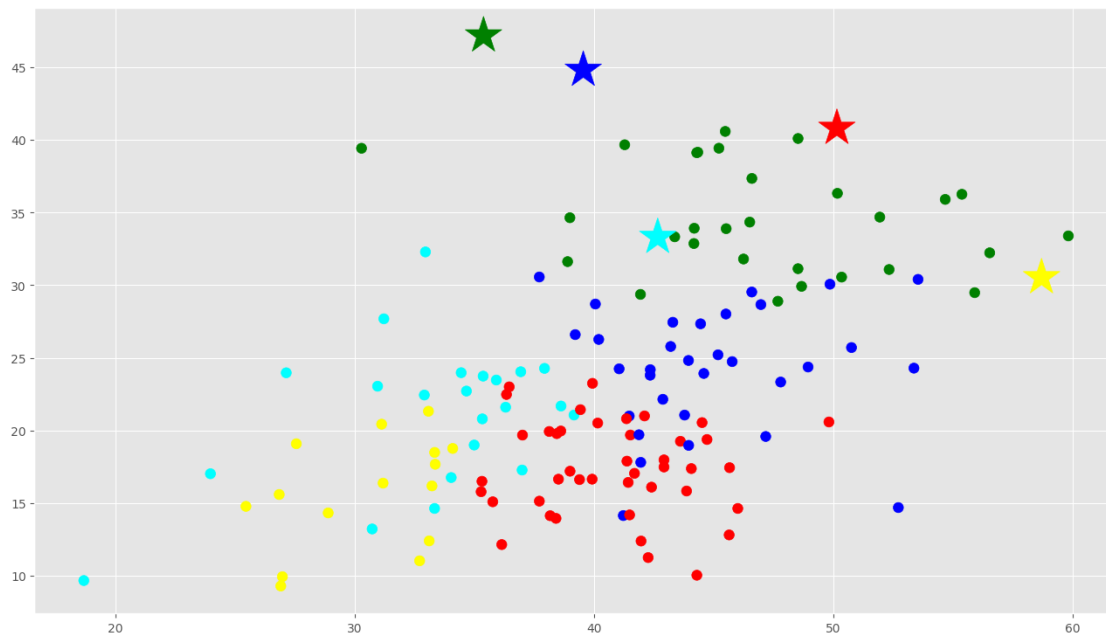
```
[26]: f1 = data['op'].values
      f2 = data['ag'].values
      plt.scatter(f1,f2, c=asignar, s=70)
      plt.scatter(C[:,0], C[:,1], marker="*", c=colores, s=1000)
```

```
[26]: <matplotlib.collections.PathCollection at 0x31db5ce80>
```



```
[27]: f1 = data['ex'].values
      f2 = data['ag'].values
      plt.scatter(f1,f2, c=asignar, s=70)
      plt.scatter(C[:,0], C[:,1], marker="*", c=colores, s=1000)
```

```
[27]: <matplotlib.collections.PathCollection at 0x319db70a0>
```



```
[28]: copy = pd.DataFrame()
```

```
[29]: copy['usuario'] = data['usuario'].values
      copy['categoria'] = data['categoria'].values
      copy['label'] = labels

      copy.head()
```

```
[29]:
```

	usuario	categoria	label
0	3gerardpique	7	1
1	aguerosergiokun	7	3
2	albertochicote	4	1
3	AlejandroSanz	2	1
4	alfredocasero1	4	1

```
[31]: cantidadGrupo = pd.DataFrame()
      cantidadGrupo['color']=colores
      cantidadGrupo['cantidad']=copy.groupby('label').size()
```

```
cantidadGrupo
```

```
[31]:
```

	color	cantidad
0	red	42
1	green	29
2	blue	32
3	cyan	22
4	yellow	15

```
[32]: group_referrer_index = copy['label'] ==0
group_referrals = copy[group_referrer_index]
diversidadGrupo = pd.DataFrame()
diversidadGrupo['categoria']=[0,1,2,3,4,5,6,7,8,9]
diversidadGrupo['cantidad']=group_referrals.groupby('categoria').size()
diversidadGrupo
```

```
[32]:
```

	categoria	cantidad
0	0	NaN
1	1	11.0
2	2	15.0
3	3	6.0
4	4	3.0
5	5	1.0
6	6	2.0
7	7	2.0
8	8	1.0
9	9	1.0

```
[34]: closest, _ = pairwise_distances_argmin_min(kmeans.cluster_centers_, X)
closest
```

```
[34]: array([ 21, 121, 107,  91,  82])
```

```
[35]: users = data['usuario'].values
for row in closest:
    print(users[row])
```

```
carmenelectra
SerranoIsmael
Pablo_Iglesias_
kobe Bryant
JudgeJudy
```

```
[36]: X_new = np.array([[45.92,57.74,15.66]])
new_labels = kmeans.predict(X_new)
print(new_labels)
```

```
[2]
```

[]: